

Grafana - Cheat Sheet

Grafana is the visualization layer for time-series data, logs, and traces. Built originally as a fork of Kibana 3 in 2014, it's now the de facto frontend for Prometheus, Loki, and most modern observability backends.

The model: connect data sources, build panels with queries, arrange panels into dashboards, share. One Grafana instance can query Prometheus, BigQuery, Postgres, and CloudWatch on the same dashboard.

Architecture

A single Grafana server runs as a web app. It doesn't store metrics or logs. It queries data sources on demand and renders the results.

Component	Role
Grafana server	Web app + API. Renders dashboards, manages users.
Data source	Plugin that talks to an external system (Prometheus, Loki, Postgres, etc.)
Dashboard	JSON config with panels, variables, and layout
Panel	A single visualization with one or more queries
Grafana DB	SQLite by default. Stores users, dashboards, alerts, settings. (Postgres or MySQL for HA.)
Alerting engine	Evaluates rules, sends notifications

The metrics data itself stays in Prometheus, Loki, or whatever backend you use. Grafana is a query and rendering layer.

Data sources

Pick one per backend. A dashboard can mix data from multiple sources.

Source	Used for
Prometheus	Time-series metrics. Most common.
Loki	Logs, with label-based querying. Pairs with Prometheus.
Tempo	Distributed traces. Grafana's tracing backend.
Elasticsearch / OpenSearch	Logs, full-text search
PostgreSQL, MySQL, MSSQL	Application database for business metrics
BigQuery, Athena, Snowflake	Data warehouse for analytics
CloudWatch, Stackdriver, Azure Monitor	Cloud-provider metrics
InfluxDB, Graphite	Older time-series stores
Jaeger, Zipkin	Older tracing backends
LGTM stack	Loki, Grafana, Tempo, Mimir. Grafana's full observability set.

Add a data source via UI or provisioning file:

```
# /etc/grafana/provisioning/datasources/prometheus.yml
apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
```

Dashboards and panels

A dashboard is a collection of panels arranged in a grid. Each panel has one or more queries.

Panel types

Type	Best for
Time series	Line, area, or bar over time. The default.
Stat	A single number, often with trend
Gauge	One value vs a min/max range
Bar gauge	Many values vs a range (CPU per core, latency per service)
Bar chart	Categorical totals
Pie chart	Composition. Use sparingly.
Table	Rows of values, sortable, filterable
Heatmap	Distribution over time (latency histograms)
Histogram	Distribution of values
Logs	Log lines, paired with Loki or Elasticsearch
Traces	Distributed trace view (Tempo, Jaeger)
Geo map	Geographic distribution
Node graph	Service dependency graph
Text	Markdown notes or instructions

For most backend services, you want: time series for rates and latency, stat for current values, table for top-N lists, logs for raw events, heatmap for latency distribution.

Query editor

Each panel has a query editor matching the data source.

For Prometheus:

```
sum by (status) (rate(http_requests_total{job="$service"}[$__rate_interval]))
```

`$service` is a variable (see below). `$__rate_interval` is a built-in that picks a window matching the dashboard refresh.

For Loki:

```
sum by (level) (rate({job="$service"} | json | level != "" [5m]))
```

For SQL:

```
SELECT
  $__timeGroup(created_at, '5m') AS time,
  COUNT(*) AS orders
FROM orders
WHERE $__timeFilter(created_at)
GROUP BY 1
ORDER BY 1
```

`$__timeGroup` and `$__timeFilter` are Grafana macros that respect the dashboard time range.

Transformations

Reshape query results before plotting. Useful when you can't get the right shape from the query.

Transformation	Use
Group by	Aggregate rows by a column
Filter by name	Drop columns
Filter by value	Drop rows
Join	Combine two query results on a key
Calculate field	Add a computed column
Organize fields	Rename or reorder
Rows to fields	Pivot

Apply in the panel's Transform tab. Reasonable for last-mile cleanup. Heavy logic should live in the query.

Variables (template variables)

Make dashboards reusable across services, environments, or instances. Defined in dashboard settings.

Variable type	Source
Query	From a query against a data source (e.g., all <code>job</code> labels in Prometheus)
Custom	Hardcoded list
Text box	Free-form user input
Constant	Hidden, fixed value
Data source	Switch between data sources
Interval	Time interval picker
Ad hoc filters	Per-panel label filter

Example: a `$service` variable populated from Prometheus.

```
Query: label_values(http_requests_total, job)
```

Then in any panel:

```
rate(http_requests_total{job="$service"}[5m])
```

Multi-value variables: enable “Include All option” and use the regex form:

```
rate(http_requests_total{job=~"$service"}[5m])
```

When the user picks “All”, `$service` becomes the regex `.*` and the query matches everything.

Built-in variables

Variable	What it gives
<code>\$__interval</code>	The auto-calculated step for the current zoom level
<code>\$__rate_interval</code>	Like <code>\$__interval</code> but at least 4 scrape intervals (good default for <code>rate</code>)
<code>\$__range</code>	Total dashboard time range as a duration
<code>\$__from</code> , <code>\$__to</code>	Start and end timestamps
<code>\$__user.login</code>	Currently logged-in user
<code>\$__org.name</code>	Current org

Annotations

Marks on a time series. Useful for marking deploys, incidents, feature flag flips.

```
SELECT
  deployed_at AS time,
  version AS text,
  service AS tags
FROM deploys
WHERE $__timeFilter(deployed_at)
```

Configure once per dashboard. Annotations show up as vertical lines on every time-series panel. You can correlate “what changed” with “what broke” at a glance.

Alerting (unified alerting)

Grafana 8+ ships a unified alerting model. Alerts live next to dashboards but evaluate independently.

Alert rule structure

```
Folder: prod
Rule group: orders

Rule: HighErrorRate
  Query A (Prometheus):
    sum(rate(http_requests_total{status=~"5..",job="orders"}[5m]))
    / sum(rate(http_requests_total{job="orders"}[5m]))
  Condition:
    last() of A > 0.01 for 5m
  Labels:
    severity = critical
    team = platform
  Annotations:
    summary = "Error rate above 1% on orders"
```

Contact points handle notifications: Slack, PagerDuty, OpsGenie, email, webhook, generic HTTP. Routing policies decide which alerts go to which contact point based on labels.

Alert states:

- **Normal:** condition not met.
- **Pending:** condition met but not long enough.
- **Firing:** condition held for `for` duration. Notification sent.
- **No data:** query returned nothing. Treat as you configure.

The unified alerting model replaced the old per-panel alerts. New dashboards should use unified alerting.

Grafana alerting vs Prometheus alerting

Both work. Grafana alerting is centralized and handles multiple data sources. Prometheus alerting lives close to the metrics and survives even if Grafana is down. Many teams keep Prometheus rules as the source of truth for critical alerts and use Grafana for dashboards plus less-critical alerts.

Dashboards as code

Hand-clicked dashboards are easy to lose and hard to review. Provision them from JSON or a code generator.

File provisioning

```
# /etc/grafana/provisioning/dashboards/all.yml
apiVersion: 1
providers:
  - name: default
    folder: Services
    type: file
    options:
      path: /var/lib/grafana/dashboards
```

Drop JSON dashboards in the path. Grafana picks them up on start.

Generators

Tool	Notes
Grafonnet	Jsonnet library for building dashboards
Grafanalib	Python library
Terraform Grafana provider	Manage dashboards as Terraform resources
Grafana Cloud Dashboard API	REST API for create/update

The dashboard JSON itself is awkward to write by hand. Generators produce it from a smaller DSL.

Versioning

Even without a generator, export the dashboard JSON and commit it. Use the API or the “Save to file” export. The “Share” link gives a different format that won’t reimport cleanly.

JSON model

A dashboard is just JSON. Understanding the structure helps when scripting or debugging.


```

{
  "title": "Orders service",
  "uid": "orders-svc",
  "tags": ["orders", "prod"],
  "timezone": "browser",
  "schemaVersion": 38,
  "templating": {
    "list": [ /* variables */ ]
  },
  "panels": [
    {
      "id": 1,
      "type": "timeseries",
      "title": "Request rate",
      "gridPos": { "x": 0, "y": 0, "w": 12, "h": 8 },
      "targets": [
        { "expr": "sum(rate(http_requests_total[5m]))", "refId": "A" }
      ]
    }
  ]
}

```

Key fields: `uid` (stable ID, use it for cross-dashboard links), `panels` (array), `templating.list` (variables), `gridPos` (24-column grid).

Folders, permissions, teams

Concept	What it is
Org	Top-level tenant. Most setups use one.
User	A person with a login
Team	A group of users for permission grants
Folder	Holds dashboards. Permissions inherit from the folder.
Permission	Viewer, Editor, Admin per dashboard or folder

For larger setups: one folder per team, default permission `Viewer` for the org, `Editor` for the owning team.

Plugins

Three types:

- **Panel plugins** (new visualization types).
- **Data source plugins** (connect to a new backend).
- **App plugins** (bundles of dashboards, data sources, and panels).

Install via UI, CLI (`grafana-cli plugins install ...`), or environment variable in a container. Signed plugins are required by default in production.

Grafana vs Kibana

Both visualize. Different sweet spots.

Aspect	Grafana	Kibana
Primary backend	Anything	Elasticsearch
Strength	Multi-source dashboards, time-series	Log search and exploration
Alerting	Unified, multi-source	Tied to Elasticsearch
Pricing	Open source + Grafana Cloud	Free with Elastic / OpenSearch
Common pairing	Prometheus + Loki + Tempo	Elasticsearch + Logstash + Beats

Many teams run both: Grafana for metrics dashboards, Kibana for log exploration in Elasticsearch.

Best practices

- **Use variables** so one dashboard covers many services or environments. Cuts the dashboard count by an order of magnitude.
- **Use `$__rate_interval`** for `rate()` queries on Prometheus. Avoids breakage when the user zooms in past the scrape window.
- **Commit dashboards to git.** Use the API or “Save to file” export.
- **One dashboard per service or per concern.** Service-level overview, infra detail, business metrics. Don’t dump everything into one mega-dashboard.
- **Keep panel count under 25.** More than that and rendering slows, queries pile up, the page locks during refresh.
- **Annotate deploys.** A deploy at 10:03 next to an error rate spike at 10:04 makes the cause obvious.
- **Use folders and permissions** to keep teams from clobbering each other’s dashboards.
- **Pick consistent units.** Bytes, seconds, percent. Wrong units mislabel everything.

- **Refresh interval matters.** A 5-second refresh on a heavy dashboard makes Grafana and the backend sweat.
 - **Use template variables.** Dashboards survive renames and label changes that would break hardcoded queries.
-

Common gotchas

- **rate() on small windows.** A `rate(metric[1m])` at a 1-minute scrape interval gets only one or two samples. Use `$__rate_interval`.
 - **Wrong units on stat panels.** A duration shown as seconds-without-formatting looks like an enormous number. Set the unit explicitly.
 - **\$service=All regex behavior.** When using multi-select, the variable becomes `(svc1|svc2|svc3)`, which doesn't match anything unless your query uses `=~`. Always use `=~` for multi-value variables.
 - **Browser caching dashboards.** A change saved on one tab may not appear on another. Hard refresh.
 - **Dashboard JSON drift.** Hand-editing in the UI and saving overwrites your provisioned JSON. Read-only mode for production.
 - **Plugin signature failures.** Production Grafana refuses unsigned plugins by default. Set `allow_loading_unsigned_plugins` only if you trust the source.
 - **Variable explosion.** A query variable that returns 1000 services makes the dropdown unusable. Use regex filters or scope by another variable.
 - **Annotations from heavy queries.** An annotation query that runs every refresh slows the page. Cache or limit.
 - **Default time range.** A dashboard that defaults to "last 5 minutes" is useless for incident review. Pick something sensible.
 - **Public dashboards leaking data.** Public dashboards (Grafana 10+) skip auth. Don't expose ones that show internal data.
 - **Heatmap rendering.** Heatmaps need histograms with proper buckets. Without them, the visualization is garbage.
 - **Alerting on the wrong query.** A panel showing average doesn't catch a tail spike. Alerts should use the same metrics as the panel but often need different aggregations.
-

Quick reference

Concept	Key fact
Data source	Connects Grafana to a backend (Prometheus, Loki, etc.)
Panel	One visualization with one or more queries
Dashboard	A collection of panels, plus variables
Variable	Reusable parameter (<code>\$service</code> , <code>\$env</code>)
<code>\$__rate_interval</code>	Auto-sized window for <code>rate</code> queries
<code>\$__timeFilter</code>	SQL macro for the dashboard time range
Transformation	Last-mile reshape after the query
Annotation	Marks on time-series panels (deploys, incidents)
Unified alerting	Centralized rules, evaluated across data sources
Contact point	Where alerts go (Slack, PagerDuty, etc.)
Provisioning	Dashboards and data sources from config files
Grafonnet / Grafanalib	DSL or library for dashboards-as-code
Folder	Permission boundary for dashboards
Plugin types	Panel, data source, app
LGTM stack	Loki, Grafana, Tempo, Mimir. Grafana's full observability set.